

Nombre: Jorge Manuel Oyoqui Aguilera
Matrícula: A01711783
Fecha: 26/05/2025
Materia: Construcción de Software y Toma de Decisiones
Grupo: 501
Profesores: Enrique Alfonso Calderón Balderas,
Denisse L. Maldonado Flores, Alejandro Fernández Vilchis



Laboratorio 28. Triggers

Trigger 1: Guardar acciones dentro de la tabla Historial Actividades

Automatizando finalmente esta tabla (que actualmente en el sistema se debe de hacer de forma manual), diseñé técnicamente 3 triggers donde se guarda dentro de la tabla de Historial Actividades cualquier actividad que haga un usuario dentro de la tabla de Productos, como, por ejemplo: añadir, editar o eliminar productos, guardando información como qué acción se hizo, qué usuario hizo la acción y a qué hora lo hizo. Mi script es el siguiente:

#Trigger 1: Guardar acciones en la tabla Historial Actividades

-- Para mayor seguridad, me aseguro de que ID_Usuario sea un campo obligatorio
ALTER TABLE CuentaFacil.Historial_Actividades **MODIFY** ID_Usuario **INT NOT NULL**;

-- Creo el trigger para INSERT
CREATE TRIGGER log_product_insert
AFTER INSERT ON CuentaFacil.Productos
FOR EACH ROW
BEGIN
 INSERT INTO CuentaFacil.Historial_Actividades (Tipo_Accion,
 Descripcion_Accion, FechaYHora_Actividad, ID_Usuario)
 VALUES ('INSERT', **CONCAT**('Se añadió el producto ',
 NEW.Nombre_Producto, ' con ID ', **NEW.ID_Producto**), **NOW**()),
 NEW.ID_Empresa);
END

-- Creo el trigger para UPDATE
CREATE TRIGGER log_product_update
AFTER UPDATE ON CuentaFacil.Productos
FOR EACH ROW

BEGIN

INSERT INTO CuentaFacil.Historial_Actividades (Tipo_Accion,
Descripcion_Accion, FechaYHora_Actividad, ID_Usuario)

VALUES ('UPDATE', **CONCAT**('Se actualizó el producto ',
NEW.Nombre_Producto, ' con ID ', **NEW**.ID_Producto), **NOW**() ,
NEW.ID_Empresa);

END

-- Creo el trigger para DELETE

CREATE TRIGGER log_product_delete

AFTER DELETE ON CuentaFacil.Productos

FOR EACH ROW

BEGIN

INSERT INTO CuentaFacil.Historial_Actividades (Tipo_Accion,
Descripcion_Accion, FechaYHora_Actividad, ID_Usuario)

VALUES ('DELETE', **CONCAT**('Se eliminó el producto ',
OLD.Nombre_Producto, ' con ID ', **OLD**.ID_Producto), **NOW**() ,
OLD.ID_Empresa);

END

Y, para probar mis triggers, corrí los siguientes scripts:

-- Inserto un nuevo producto

INSERT INTO CuentaFacil.Productos (ID_Producto, Nombre_Producto,
Cantidad_Inicial, Cantidad_Vendida, Cantidad_Restante, Costo_Pieza,
Total_Inventario, Fecha_Modificacion, ID_Empresa)

VALUES (20, 'Anillo de dragón de diamantes', 50, 0, 50, 12000, 600000, **NOW**() ,
1);

Resultado: Se agrega correctamente el nuevo producto de nombre Anillo de dragón de diamantes a la tabla de Prodcutos.

Propiedades Datos Diagrama								
historial_actividades Enter a SQL expression to filter results (use Ctrl+Space)								
	ID_Producto	Nombre_Producto	Cantidad_Inicial	Cantidad_Vendida	Cantidad_Restante	Costo_Pieza	Total_Inventario	
1	1	Collar Pandora	25	12	13	350	8.75	
2	2	Anillo de Plata	42	13	29	45	7.95	
3	3	Anillo de Oro	13	4	9	125	14	
4	4	Collar de Jade	54	6	48	70	23	
5	5	Pulsera de Piel	89	54	35	35	10.89	
6	6	Aretes de Diamante	5	1	4	2.500	12	
7	7	Aretes de Oro Rosa	15	4	11	1.650	23	
8	8	Anillo con Rubíes	15	1	14	1.650	18	
9	9	Collar de Cadenas	35	21	14	35	5.79	
10	10	Collar de Perlas	17	12	5	135	32	
11	11	Collar de platino	15	5	15	35	52	
12	12	Collar de diamantino	50	10	50	50	2.50	
13	13	Collar de Zafiros	50	15	35	5.500	82.50	
14	15	Reloj de Plata	100	35	65	8.000	280.00	
15	16	Aretes de diamante	30	5	25	10.000	50.00	
16	17	Colla de cadenas de oro	200	85	115	3.500	297.50	
17	18	Anillo de Calavera	150	60	90	150	9.00	
18	20	Anillo de dragón de diamantes	50	0	50	12.000	600.00	

Y en la tabla de Historial Actividades se guarda la acción realizada por el usuario 1 de añadir un nuevo producto:

Propiedades Datos Diagrama						
historial_actividades Enter a SQL expression to filter results (use Ctrl+Space)						
	ID_Actividad	Tipo_Accion	Descripcion_Accion	FechaYHora_Actividad	ID	
32	32	Salida de sesión	Usuario Regina Villalobos realizó la ac	[NULL]		
33	33	Edición de archivo	Usuario Adriana Calderón editó el arch	[NULL]		
34	34	Eliminación de archivo	Usuario Valeria Rivas realizó la acción:	[NULL]		
35	35	Eliminación de archivo	Usuario Regina Villalobos realizó la ac	[NULL]		
36	36	Subida de archivo	Usuario Carlos Mancilla realizó la acci	[NULL]		
37	37	Descarga de archivo	Usuario Renata Lozano descargó el arc	[NULL]		
38	38	Salida de sesión	Usuario Hugo Salgado realizó la acció	[NULL]		
39	39	Edición de archivo	Usuario Lucia Montes editó el archivo:	[NULL]		
40	40	Eliminación de archivo	Usuario Nadia Cervantes realizó la acci	[NULL]		
41	41	Subida de archivo	Usuario Regina Villalobos realizó la ac	[NULL]		
42	42	Descarga de archivo	Usuario Regina Villalobos descargó el	[NULL]		
43	43	Subida de archivo	Usuario Luis Fernando Neri realizó la a	[NULL]		
44	44	Salida de sesión	Usuario Diana Treviño realizó la acció	[NULL]		
45	45	Descarga de archivo	Usuario Luis Fernando Neri descargó e	[NULL]		
46	46	Cambio de contraseña	Usuario Renata Lozano realizó la acció	[NULL]		
47	47	Edición de archivo	Usuario Bruno Castañeda editó el archi	[NULL]		
48	48	Cambio de contraseña	Usuario Isabel Coronel realizó la acció	[NULL]		
49	49	Subida de archivo	Usuario Emilio Vargas realizó la acción	[NULL]		
50	50	Descarga de Archivo	Usuario Valeria Rivas descargó el archi	2025-05-14 20:13:00.000		
51	52	Registro empresa	Se agregó una nueva empresa	2025-05-16 20:53:34.000		
52	53	Cambio de datos	Usuario modificó su perfil	2025-05-16 20:53:34.000		
53	54	Eliminación cuenta	Se eliminó una cuenta	2025-05-16 20:53:34.000		
54	55	Exportación datos	Usuario descargó un informe	2025-05-16 20:53:34.000		
55	56	INSERT	Se añadió el producto Anillo de dragó	2025-05-26 17:09:29.000		

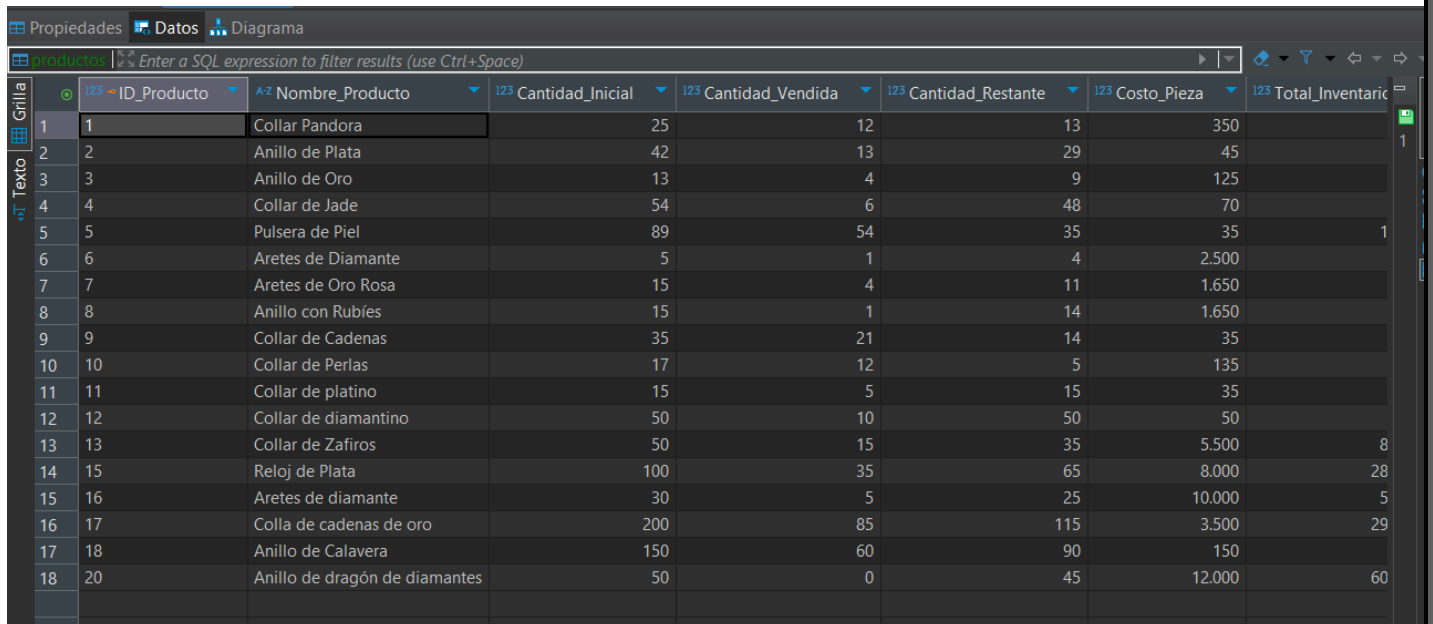
-- Modifico el producto (cambio la cantidad restante)

UPDATE CuentaFacil.Productos

SET Cantidad_Restante = 45, Fecha_Modificacion = **NOW()**

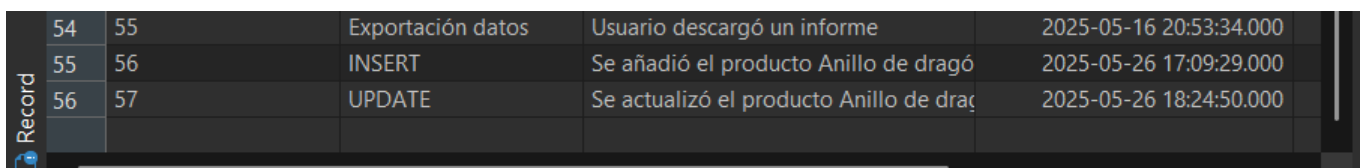
WHERE ID_Producto = 20;

Resultado: Modifico el último producto que agregamos usando su ID (en este caso, el anillo de dragón de diamante) para modificar su cantidad restante de 50 a 45, actualizando de paso la fecha de modificación.



	ID_Producto	Nombre_Producto	Cantidad_Inicial	Cantidad_Vendida	Cantidad_Restante	Costo_Pieza	Total_Inventaric
1	1	Collar Pandora	25	12	13	350	
2	2	Anillo de Plata	42	13	29	45	
3	3	Anillo de Oro	13	4	9	125	
4	4	Collar de Jade	54	6	48	70	
5	5	Pulsera de Piel	89	54	35	35	1
6	6	Aretes de Diamante	5	1	4	2.500	
7	7	Aretes de Oro Rosa	15	4	11	1.650	
8	8	Anillo con Rubíes	15	1	14	1.650	
9	9	Collar de Cadenas	35	21	14	35	
10	10	Collar de Perlas	17	12	5	135	
11	11	Collar de platino	15	5	15	35	
12	12	Collar de diamantino	50	10	50	50	
13	13	Collar de Zafiros	50	15	35	5.500	8
14	15	Reloj de Plata	100	35	65	8.000	28
15	16	Aretes de diamante	30	5	25	10.000	5
16	17	Colla de cadenas de oro	200	85	115	3.500	29
17	18	Anillo de Calavera	150	60	90	150	
18	20	Anillo de dragón de diamantes	50	0	45	12.000	60

Y la tabla de Historial Actividades también se actualiza con esta nueva acción:



54	55	Exportación datos	Usuario descargó un informe	2025-05-16 20:53:34.000
55	56	INSERT	Se añadió el producto Anillo de dragón	2025-05-26 17:09:29.000
56	57	UPDATE	Se actualizó el producto Anillo de dragón	2025-05-26 18:24:50.000

Y, por último:

-- Elimino el producto

DELETE FROM CuentaFacil.Productos **WHERE** ID_Producto = 20;

Resultado: Se elimina el objeto de anillo de dragón de diamantes, de modo que desaparece de la tabla de productos, pero se actualiza la tabla de historial actividades mostrando la acción de borrar ese producto.

Así que, como se ve en la tabla, el producto se borró.

Enter a SQL expression to filter results (use Ctrl+Space)									
	123 ID_Producto	A2 Nombre_Producto	123 Cantidad_Inicial	123 Cantidad_Vendida	123 Cantidad_Restante	123 Costo_Pieza	123 Total_Inventari		
1	1	Collar Pandora	25	12	13	350			1
2	2	Anillo de Plata	42	13	29	45			
3	3	Anillo de Oro	13	4	9	125			
4	4	Collar de Jade	54	6	48	70			
5	5	Pulsera de Piel	89	54	35	35			1
6	6	Aretes de Diamante	5	1	4	2.500			
7	7	Aretes de Oro Rosa	15	4	11	1.650			
8	8	Anillo con Rubíes	15	1	14	1.650			
9	9	Collar de Cadenas	35	21	14	35			
10	10	Collar de Perlas	17	12	5	135			
11	11	Collar de platino	15	5	15	35			
12	12	Collar de diamantino	50	10	50	50			
13	13	Collar de Zafiros	50	15	35	5.500			8
14	15	Reloj de Plata	100	35	65	8.000			28
15	16	Aretes de diamante	30	5	25	10.000			5
16	17	Colla de cadenas de oro	200	85	115	3.500			29
17	18	Anillo de Calavera	150	60	90	150			

Pero la tabla de historial actividades de actualizó con la acción:

53	54	Eliminación cuenta	Se eliminó una cuenta	2025-05-16 20:53:34.000	4
54	55	Exportación datos	Usuario descargó un informe	2025-05-16 20:53:34.000	5
55	56	INSERT	Se añadió el producto Anillo de dragó	2025-05-26 17:09:29.000	1
56	57	UPDATE	Se actualizó el producto Anillo de dragó	2025-05-26 18:24:50.000	1
57	58	DELETE	Se eliminó el producto Anillo de dragó	2025-05-26 18:28:52.000	1

Trigger 2: Control de cantidad de pedidos

Donde se define que, a la hora de crear un pedido, no se pueda registrar un pedido con una cantidad de productos menor a 1. El script del trigger fue:

```
#Trigger 2: Control de cantidad de pedidos
-- Evita que los pedidos se registren con una cantidad menor a 1. De modo que si
alguien lo intenta, lanza un error.
CREATE TRIGGER prevent_zero_quantity
BEFORE INSERT ON CuentaFacil.Incluye
FOR EACH ROW
BEGIN
    IF NEW.Cantidad_Individual_Producto < 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: No se pueden registrar pedidos con cantidad
de producto menor a 1';
    END IF;
end
```

Y, para probarlo, generé el siguiente script para probarlo, donde el primer script es un pedido con cero productos, de modo que se debe devolver un error 45000 indicando que el pedido no es válido debido a su nula cantidad de productos. Y el segundo script ya contiene una cantidad válida de productos, de modo que el pedido sí se debería guardar.

```
-- Intento registrar un pedido con cantidad inválida

INSERT INTO CuentaFacil.Incluye (ID_Pedido, ID_Producto,
Cantidad_Individual_Producto)

VALUES (200, 101, 0); -- Esto debería fallar mandando un error 45000 con un
mensaje que diga que la cantidad NO debe ser menor a 1
```

Resultado: Maria DB me marca un error, pues no se pueden subir pedidos con cero productos, marcándome específicamente un error 45000.

```

#Pruebas sobre el trigger 2
-- Intento registrar un pedido con cantidad inválida
INSERT INTO CuentaFacil.Incluye (ID Pedido, ID Producto, Cantidad Individual Product
VALUES (5, 10, 0); -- Esto debería fallar mandando un error 45000 con un mensaje que

-- Pruebo con una cantidad válida
INSERT INTO CuentaFacil.Incluye (ID Pedido, ID Producto, Cantidad Individual Product
VALUES (5, 10, 5); -- Esto se debería registrar correctamente

```

Estadísticas 1 x

SQL Error [1644] [45000]: (conn=5) Error: No se pueden registrar pedidos con cantidad de producto menor a 1

-- Intento registra
INSERT INTO CuentaF
VALUES (5, 10, 0);

Y cuando inserto un pedido con una cantidad correcta de productos:

-- Pruebo con una cantidad válida

```
INSERT INTO CuentaFacil.Incluye (ID_Pedido, ID_Producto,
Cantidad_Individual_Producto)
```

```
VALUES (5, 10, 5); -- Esto se debería registrar correctamente
```

Resultado: El nuevo pedido se agrega en la tabla de Incluye (pues es la que relaciona al pedido con el producto), sólo que en este caso no se añade hasta el fondo de la tabla, sino en una posición entre medias según el ID del pedido, de modo que decidí resaltarla para mostrar de mejor manera que sí se agregó:

Propiedades Datos Diagrama				
incluye Enter a SQL expression to filter results (use Ctrl+Space)				
	ID_Pedido	ID_Producto	Cantidad_Individual_Producto	
1	1	1	12	
2	2	2	4	
3	3	2	20	
4	3	3	5	
5	4	4	1	
6	5	5	6	
7	5	10	5	
8	6	6	7	
9	7	7	2	

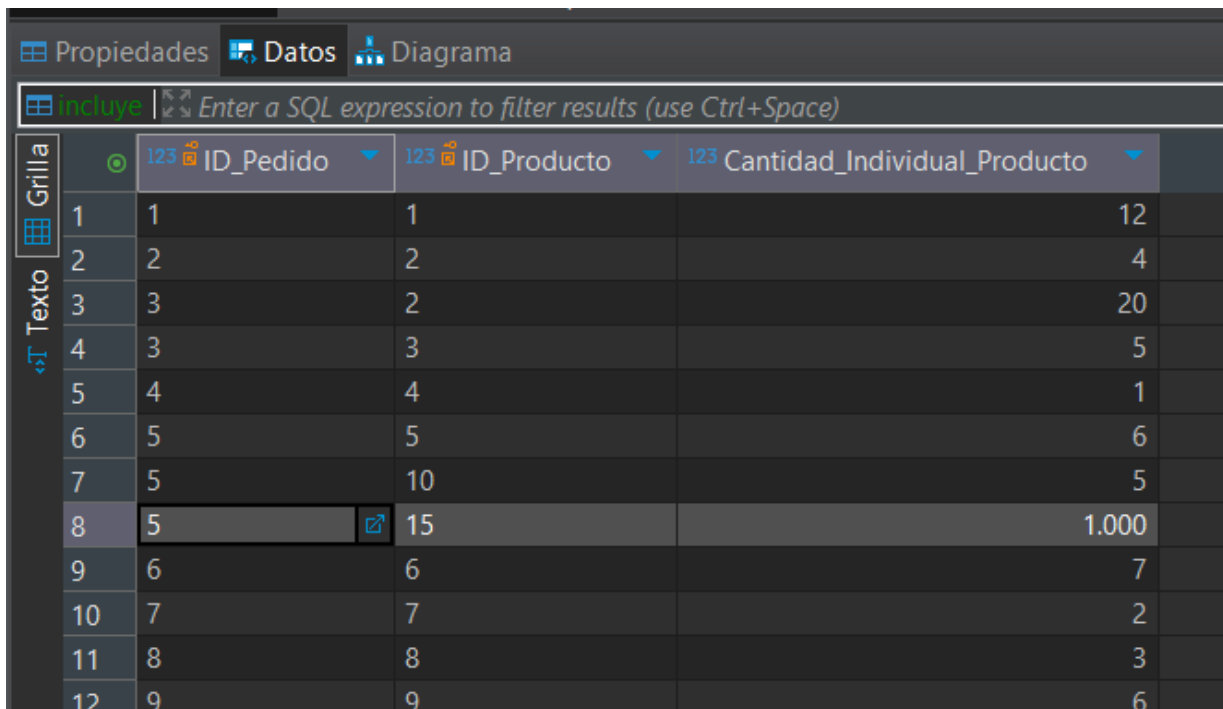
Ahora, por si no pareciera verídico, generé el siguiente código con datos más exagerados para que fuera notable:

-- Pruebo con una cantidad válida

```
INSERT INTO CuentaFacil.Incluye (ID_Pedido, ID_Producto, Cantidad_Individual_Producto)
```

```
VALUES (5, 15, 1000); -- Esto se debería registrar correctamente
```

Como se podrá notar, anteriormente sólo habían dos pedidos con el ID de 5, pero ahora hay un tercer pedido con el ID de pedido 5 que está solicitando 1000 productos del tipo del producto 15:



	123 ID_Pedido	123 ID_Producto	123 Cantidad_Individual_Producto
1	1	1	12
2	2	2	4
3	3	2	20
4	3	3	5
5	4	4	1
6	5	5	6
7	5	10	5
8	5	15	1.000
9	6	6	7
10	7	7	2
11	8	8	3
12	9	9	6

Y esos serían mis dos triggers. El primero sirve para automatizar la documentación de los cambios que los usuarios (de tipo administrador o empresa, pero con sus respectivos productos) hacen sobre los datos en la base de datos; y el segundo sirve para evitar errores incongruentes como tener un pedido de 0 productos. De este modo ambos resultan bastante útiles para el sistema, ya sea por su documentación como para el manejo de errores.

Mi Script

Ahora, comparto **TODO** mi script por si quisiera hacer directamente un copy paste para probarlo:

#RECOMENDACION: Para evitar problemas, recomiendo correr cada código para crear los triggers de forma individual (pues así le hice yo)

#lo mismo para los ejemplos. Corre una parte a la vez para revisar mejor los resultados.

#Trigger 1: Guardar acciones en la tabla Historial Actividades

-- Para mayor seguridad, me aseguro de que ID_Usuario sea un campo obligatorio

ALTER TABLE CuentaFacil.Historial_Actividades **MODIFY** ID_Usuario **INT NOT NULL**;

-- Creo el trigger para INSERT

CREATE TRIGGER log_product_insert

AFTER INSERT ON CuentaFacil.Productos

FOR EACH ROW

BEGIN

INSERT INTO CuentaFacil.Historial_Actividades (Tipo_Accion, Descripcion_Accion, FechaYHora_Actividad, ID_Usuario)

VALUES ('INSERT', **CONCAT**('Se añadió el producto ', **NEW**.Nombre_Producto, ' con ID ', **NEW**.ID_Producto), **NOW**(), **NEW**.ID_Empresa);

END

-- Creo el trigger para UPDATE

CREATE TRIGGER log_product_update

AFTER UPDATE ON CuentaFacil.Productos

FOR EACH ROW

BEGIN

INSERT INTO CuentaFacil.Historial_Actividades (Tipo_Accion, Descripcion_Accion, FechaYHora_Actividad, ID_Usuario)

VALUES ('UPDATE', **CONCAT**('Se actualizó el producto ', **NEW**.Nombre_Producto, ' con ID ', **NEW**.ID_Producto), **NOW**(), **NEW**.ID_Empresa);

END

-- Creo el trigger para DELETE

CREATE TRIGGER log_product_delete

AFTER DELETE ON CuentaFacil.Productos

FOR EACH ROW

BEGIN

INSERT INTO CuentaFacil.Historial_Actividades (Tipo_Accion, Descripcion_Accion, FechaYHora_Actividad, ID_Usuario)

VALUES ('DELETE', **CONCAT**('Se eliminó el producto ', **OLD**.Nombre_Producto, ' con ID ', **OLD**.ID_Producto), **NOW**(), **OLD**.ID_Empresa);

END

#Trigger 2: Control de cantidad de pedidos

-- Evita que los pedidos se registren con una cantidad menor a 1. De modo que si alguien lo intenta, lanza un error.

CREATE TRIGGER prevent_zero_quantity

BEFORE INSERT ON CuentaFacil.Incluye

FOR EACH ROW

BEGIN

IF NEW.Cantidad_Individual_Producto < 1 **THEN**

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Error: No se pueden registrar pedidos con cantidad de producto menor a 1';

END IF;

end

#Scripts para probar los triggers

#Pruebas sobre el trigger 1

-- Inserto un nuevo producto

INSERT INTO CuentaFacil.Productos (ID_Producto, Nombre_Producto, Cantidad_Inicial, Cantidad_Vendida, Cantidad_Restante, Costo_Pieza, Total_Inventario, Fecha_Modificacion, ID_Empresa)

VALUES (20, 'Anillo de dragón de diamantes', 50, 0, 50, 12000, 600000, **NOW()**, 1);

-- Modifico el producto (cambio la cantidad restante)

UPDATE CuentaFacil.Productos

SET Cantidad_Restante = 45, Fecha_Modificacion = **NOW()**

WHERE ID_Producto = 20;

-- Elimino el producto

DELETE FROM CuentaFacil.Productos **WHERE** ID_Producto = 20;

-- Consulto el historial de actividades para verificar los registros del trigger (se recomienda correr por cada prueba ya sea de INSERT, UPDATE o DELETE para ver los resultados)

SELECT * FROM CuentaFacil.Historial_Actividades **ORDER BY** FechaYHora_Actividad **DESC;**

#Pruebas sobre el trigger 2

-- Intento registrar un pedido con cantidad inválida

INSERT INTO CuentaFacil.Incluye (ID_Pedido, ID_Producto,
Cantidad_Individual_Producto)

VALUES (5, 10, 0); -- Esto debería fallar mandando un error 45000 con un mensaje que diga que la cantidad NO debe ser menor a 1

-- Pruebo con una cantidad válida

INSERT INTO CuentaFacil.Incluye (ID_Pedido, ID_Producto,
Cantidad_Individual_Producto)

VALUES (5, 15, 1000); -- Esto se debería registrar correctamente

Preguntas del laboratorio 28

1. ¿Qué utilidad tiene un trigger (ventajas)?

Como ya vimos, un trigger es un procedimiento que se almacena en la base de datos que se ejecuta automáticamente cuando se presenta cierto evento en específico dentro de la base de datos.

Las ventajas que tiene usarlo son:

- Generar automáticamente algunos valores de columna derivados, de modo que se pueden automatizar cálculos y así evitar errores humanos a la hora de llenar una tabla, por ejemplo.
- Aplicar la integridad referencial, de modo que un trigger puede ayudar a reforzar o personalizar una relación foránea con una tabla, de modo que se mantienen relaciones coherentes entre ambas tablas.
- Registro de eventos y almacenamiento de información sobre el acceso a la tabla, de modo que se puede guardar cuándo se modificó una tabla, quién la modificó y qué datos modificó.
- Auditoría, que, retomando el punto anterior, consiste en controlar cambios importantes, pero con fines más legales o de seguridad. Si nos vamos a este enfoque, se mejora la seguridad y responsabilidad en sistemas sensibles, como en los sistemas de los bancos, por ejemplo.
- Replicación sincrónica de tablas, de modo que un trigger puede copiar datos de una tabla a otra. Así se pueden mantener varias copias de los datos actualizados en tiempo real.
- Imponer autorizaciones de seguridad, evitando que se puedan realizar ciertas acciones si no se cumplen con ciertas condiciones previas, mejorando la seguridad.
- Prevenir transacciones inválidas, de modo que un trigger puede detectar intentos de acciones no válidas o lógicas incorrectas en los datos antes de que se confirmen. De este modo se mejora la coherencia e integridad de los datos.

2. ¿Tipos de triggers?

Los tipos de triggers se pueden dividir en diferentes categorías: según el momento de ejecución, según el tipo de operación o según su alcance:

- Según el momento de ejecución:
 - BEFORE: aquellos que se ejecutan antes de realizar la operación, y se usan para validar o modificar datos antes de que se inserten o cambien.
 - AFTER: aquellos que se ejecutan luego de que la operación se completó exitosamente. Se usan para registrar cambios o replicar datos.
 - INSTEAD OF: aquellas que se ejecutan en lugar de la operación original. Se usan en listas que no pueden modificarse directamente.
- Según el tipo de operación que los activa:
 - INSERT: aquellos que se activan al hacer un nuevo registro en una tabla.
 - UPDATE: se activan cuando se modifica un archivo ya existente dentro de una tabla.
 - DELETE: se activa cuando se elimina un registro de una tabla.
- Según el alcance del trigger:
 - Row-level (o por fila): se ejecuta una vez por cada fila que se vea afectada por la operación. Se usa para validaciones o para registros específicos por cada fila.
 - Statement-level (o por instrucción): aquel que se ejecuta una sola vez sin importar la cantidad de filas que se vean afectadas. Se usa para tareas más generales como actualizar una tabla.

3. ¿En qué caso no son de utilidad?

Existen varios casos en los que los triggers pueden resultar más un problema que como una solución. Entre esos casos tenemos:

- Cuando complican el mantenimiento del sistema, complicando la lógica de este. Esto puede afectar la depuración de errores o el entendimiento del flujo del sistema.
- Cuando afectan al rendimiento del sistema ralentizando las operaciones.
- Cuando la lógica del negocio debe estar en la aplicación entonces no se debe poner como triggers. Más que nada por que si las reglas de negocio llegan a cambiar constantemente y son complejas, entonces conviene más trabajarlas, pero desde el código de la aplicación o página.

- Cuando generan dependencias ocultas. Por ejemplo, que al modificar una tabla se afecte a otra a la cual no debería de afectar. De este modo se podrían provocar errores inesperados o resultados incorrectos.
- Cuando pueden provocar bucles infinitos, causando bloqueos, uso excesivo de recursos o, en el peor de los casos, afectando directamente a la base de datos.

Referencias Bibliográficas

PostgreSQL Global Development Group (S.F.). “Chapter 37. Triggers”. Recuperado de: <https://www.postgresql.org/docs/current/triggers.html>

Microsoft (2024). “CREATE TRIGGER (Transact-SQL)”. Recuperado de: <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-trigger-transact-sql?view=sql-server-ver17>

Alura (2023). “¿Qué es y cómo y usar un trigger en SQL?”. Recuperado de: <https://www.aluracursos.com/blog/que-es-y-como-trigger-en-sql>